

TextHack: An Advanced Algorithm Engine for Typo-Tolerant Enterprise Knowledge Search

*A Project Based Learning Report Submitted in partial fulfilment of the requirements for the
award of the degree*

of

Bachelor of Technology

in The Department of Artificial Intelligence and Data Science

Data Structures and Algorithms – 3 (25CS2103E)

Submitted by

2510080006: M. Sathya Krishna

2510080001: Yashwanth

2510080005: Aneeq

2510080002: Abhinav

Under the guidance of

Anitha. P



Department of Artificial Intelligence and Data Science

Koneru Lakshmaiah Education Foundation, Aziz Nagar

Aziz Nagar – 500075

OCT - 2026.

Abstract

Finding the right document in an organisation is a string problem. Real queries contain typing mistakes, reordered words and several terms at once, and documents cite one another. A database LIKE match finds exact substrings only, so one typo returns nothing; a search library hides both the cost and the correctness of what it does. This project builds the algorithms instead, proves them, and uses them in a real product.

TextHack is a dependency-free Java engine of twenty algorithms in five families: exact and multi-pattern string matching (Knuth-Morris-Pratt, Z, Rabin-Karp, Aho-Corasick, suffix and LCP arrays), dynamic programming (Levenshtein and Damerau-Levenshtein distance, Needleman-Wunsch and Smith-Waterman alignment), network flow (Ford-Fulkerson, Edmonds-Karp, Dinic, bipartite matching), approximation (vertex cover, list and LPT scheduling) and randomised algorithms (Miller-Rabin, universal hashing, reservoir sampling). No algorithm uses `java.util`. One facade and a small query language expose the engine, and the Enterprise Knowledge Intelligence Platform compiles the same source to score every keyword search and to run a workbench page.

Six test suites make 452 assertions with no failures, including 4,000 randomised cross-checks against simple reference implementations. Benchmarks confirm the theory: KMP is 13 times faster than naive search on its worst case, and Aho-Corasick is 15.5 times faster than running KMP once per pattern for 80 patterns. In the platform, an inspection of 23 queries, including misspelled names and words that appear only inside documents, ranked the intended document first in all 23 for hybrid, fuzzy and semantic search.

Keywords: string matching, Knuth-Morris-Pratt, Aho-Corasick, suffix array, edit distance, Damerau-Levenshtein, sequence alignment, maximum flow, approximation algorithms, randomised algorithms

List of Figures

Figure 1: TextHack’s packages, its engine facade, and the parts of the platform that use it	7
Figure 2: The six-step method used for every algorithm	8
Figure 3: Aho-Corasick automaton for {he, she, his, hers}, with the links used to scan “ushers”	8
Figure 4: Citation flow: maximum flow 2 (two independent chains) equals the minimum cut $\{0 \rightarrow 1, 0 \rightarrow 2\}$	9
Figure 5: How a query is corrected and scored with KMP, Aho-Corasick and Damerau-Levenshtein.....	11
Figure 6: All six suites passing (<code>run-tests.sh</code>)	12
Figure 7: Benchmark: exact and multi-pattern search	13
Figure 8: Benchmark: suffix array and Levenshtein growth	13
Figure 9: The complexity registry in the workbench (22 algorithms)	14
Figure 10: Pattern search: Aho-Corasick matches highlighted, and the longest repeated substring	15
Figure 11: Similarity: edit distances and global and local alignments	15
Figure 12: Citation flow: Dinic’s influence and the Edmonds-Karp minimum cut.....	16
Figure 13: Hybrid search for the misspelled name “Dynmo”, found through query correction	17
Figure 14: Fuzzy search for “remte workng standrd”: three typos, the remote-working standard first.....	17

List of Tables

Table 1: Query classes and the algorithms that answer them	7
Table 2: The TextHack query language.....	10
Table 3: How TextHack scores a document for a keyword query	10
Table 4: Workbench endpoints	11
Table 5: TextHack test suites.....	12
Table 6: Benchmark results (best of 5 runs after warm-up).....	13
Table 7: Search inspection on the 315-document demo corpus (score of the first result; ✓ = intended document first)	16
Table 8: Intended document ranked first, out of 23 cases.....	16
Table 9: Challenges and how they were solved.....	18

Table of Contents

1. Introduction	6
2. Methodology	6
a. Requirements and System Design	6
b. How Each Algorithm Was Built and Verified	7
c. String Algorithms	8
d. Dynamic Programming: Edit Distance and Alignment	8
e. Network Flow for Citation Analysis	9
f. Approximation and Randomised Algorithms	9
g. Engine API and Query Language	10
h. Use in the Platform's Search	10
i. The TextHack Workbench	11
3. Experiments	11
4. Results	12
5. Conclusion and Future Work	18
References	18

TextHack: An Advanced Algorithm Engine for Typo-Tolerant Enterprise Knowledge Search

1. INTRODUCTION

Organisations keep their knowledge in thousands of documents: policies, handbooks, reports and research. Finding the right one depends on text search that copes with how people actually type. A query such as “recieve” must still find “receive”; a query of five words must check all five at once; and when documents cite each other, how strongly two of them are connected is itself a question worth answering. Each of these is a classic algorithmic problem with a known efficient solution [1], [2].

Existing approaches fall short in different ways. PostgreSQL’s LIKE and ILIKE match exact substrings only, ignore reordered words and do not rank. Full-text engines such as Lucene offer fuzzy search, but as a large external service whose algorithms are a black box, which does not meet the course’s rule of building them. Typical classroom code has its own traps: a Z-algorithm that breaks on text containing its sentinel, Rabin-Karp reporting unverified hash hits, a modular multiplication that overflows 64 bits, a Fermat primality test fooled by Carmichael numbers.

Problem statement. Build, from first principles and without library shortcuts, a verified engine of string, dynamic-programming, graph, approximation and randomised algorithms, and use it to make enterprise document search tolerant of typos, fast for many terms at once, and able to analyse citation structure, with the correctness and the complexity of every algorithm demonstrated.

Approach. Each algorithm is written against a contract, checked against a simple reference implementation on random and adversarial inputs, and measured against its complexity bound. The same verified source is then compiled into the platform’s backend, so the algorithms that pass the tests are exactly the ones that rank users’ search results.

Objectives.

- Implement the syllabus’s twenty algorithms from scratch, with no `java.util` in algorithm code.
- Prove them correct with known cases, edge cases and randomised cross-checks.
- Document and measure the complexity of every algorithm.
- Expose the engine through one facade and a small query language.
- Improve the platform’s search with KMP, Aho-Corasick and Damerau-Levenshtein.
- Let users run the algorithms on their own input through a REST API and a workbench page.

Scope. The engine covers the five algorithm families of the course. The syllabus’s Indian-language Wikipedia corpus was not imported, because it is a data-acquisition and licensing task rather than an algorithm; the engine runs instead over the platform’s documents, evaluated on a demo corpus of 315 documents. Section 2 describes the design and the algorithms, Section 3 the experiments and Section 4 the results; Section 5 concludes.

2. METHODOLOGY

a. *Requirements and System Design*

TextHack answers six classes of query, and each needs a different algorithm family (Table 1). The engine is organised as one package per family behind an engine package that holds the facade, the query language, citation-flow analysis and a complexity registry (Figure 1).

Table 1: Query classes and the algorithms that answer them

Query class	Algorithm family	Engine algorithms
Pattern search	String matching	Naive, KMP, Z, Rabin-Karp; Aho-Corasick for many patterns
Repeated text	Suffix structures	Suffix array (prefix doubling), LCP array (Kasai)
Fuzzy matching	Dynamic programming	Levenshtein; Damerau-Levenshtein, restricted (OSA) and unrestricted
Document similarity	Sequence alignment	Needleman-Wunsch (global), Smith-Waterman (local)
Citation analysis	Network flow	Ford-Fulkerson, Edmonds-Karp, Dinic; bipartite matching
Scheduling, primality	Approximation, randomisation	Vertex cover, list and LPT scheduling; Miller-Rabin, universal hashing, reservoir sampling

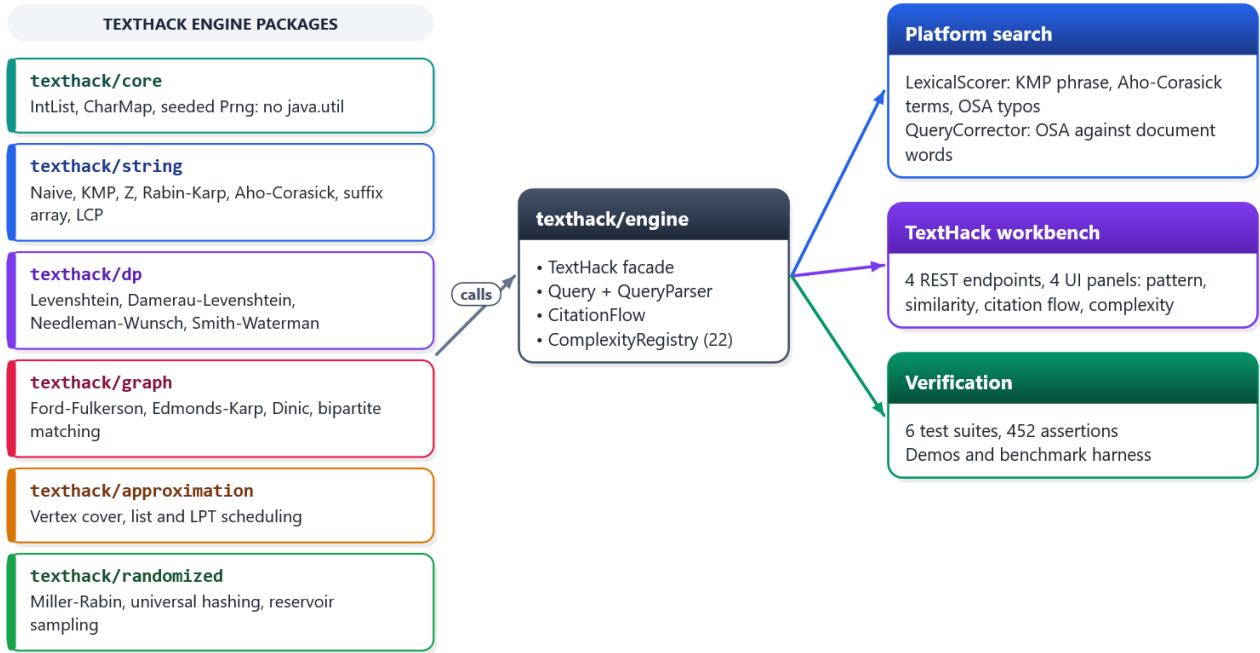


Figure 1: TextHack's packages, its engine facade, and the parts of the platform that use it

The java.util rule. Algorithm code may not use the Java collections, so the core package supplies what they would: a growable `IntList`, a sorted `CharMap` for trie children and a seeded `Prng` (`SplitMix64`). Suffixes are sorted with counting sort rather than a library sort. Because the engine depends on nothing but the JDK, the backend compiles it in directly as a second source root: there is exactly one copy of each algorithm, tested by the engine's suites and run by the platform.

b. How Each Algorithm Was Built and Verified

Every algorithm followed the same six steps (Figure 2). The reference version is deliberately simple, such as naive search or a brute-force optimum, so it is obviously correct; the efficient algorithm must agree with it on thousands of random inputs before it is trusted.

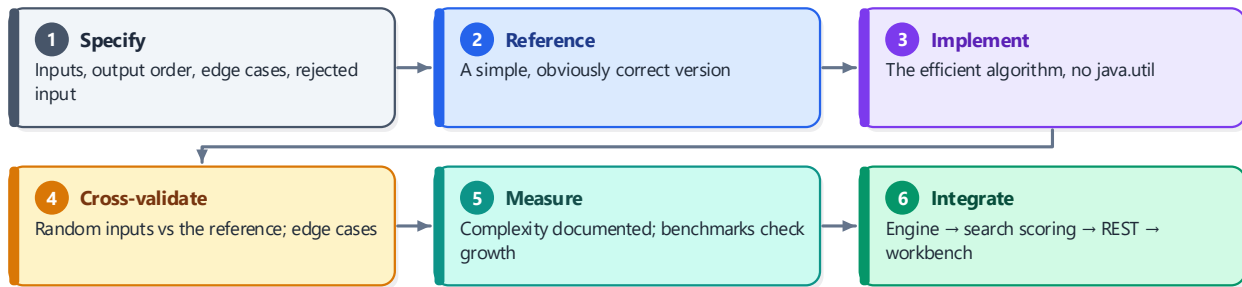
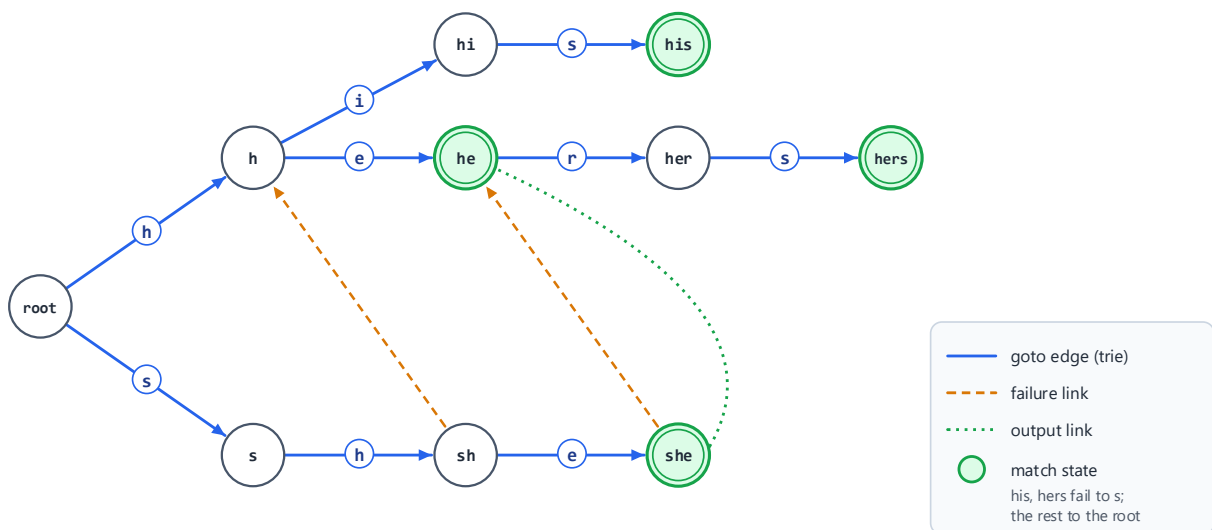


Figure 2: The six-step method used for every algorithm

c. String Algorithms

Exact matching. Knuth-Morris-Pratt [3] precomputes a failure function so the text pointer never moves back, giving $O(n + m)$ time with no pathological input. The Z-algorithm [2] is computed on the pattern alone and then matched against the text, so it needs no sentinel character and works on any text. Rabin-Karp [4] uses a rolling hash and verifies every hash hit character by character, so collisions never become false matches.

Many patterns in one pass. Aho-Corasick [5] builds a trie of all patterns, adds a failure link from each state to its longest proper suffix that is also a trie prefix, and an output link to the nearest match state reachable by failure links. The text is then scanned once, in $O(n + z)$ for z matches, however many patterns there are. Figure 3 shows the automaton for {he, she, his, hers}: scanning “ushers” reaches *she*, and the output link reports *he* at the same position, a case that broken output links miss.



Scanning “ushers” once: *she* at 1–4, *he* at 2–4 (through the output link), *hers* at 2–6.

Figure 3: Aho-Corasick automaton for {he, she, his, hers}, with the links used to scan “ushers”

Suffix structures. The suffix array is built by prefix doubling [6], ranking suffixes by their first 2^k characters with counting sort in each round, for $O(n \log n)$ time. Kasai’s algorithm [7] then computes the LCP array in $O(n)$ by carrying the common-prefix length from one suffix to the next; the largest LCP value gives the longest repeated substring.

d. Dynamic Programming: Edit Distance and Alignment

Levenshtein distance [8] counts the insertions, deletions and substitutions that turn one string into another; TextHack keeps only two rows of the table, so it needs $O(\min(n, m))$ space. Typing mistakes often swap two letters, which Levenshtein charges as two edits. Damerau-Levenshtein [9] counts an adjacent transposition as one edit, and TextHack implements both of its variants

because they give different answers: the restricted *optimal string alignment* (OSA) never edits a substring twice, while the unrestricted variant may. For “recieve the report” against “receive the reports”, Levenshtein gives 3 edits and Damerau 2.

Alignment shows *where* two texts agree. Needleman-Wunsch [10] aligns the whole of both texts and keeps the matrix for the traceback; Smith-Waterman [11] floors scores at zero to find the best matching region, which isolates a shared paragraph between two otherwise different documents. Both run in $O(n \cdot m)$ time.

e. Network Flow for Citation Analysis

Ford-Fulkerson [12] augments flow along any path in the residual graph; Edmonds-Karp [13] chooses the shortest path by breadth-first search, bounding the work by $O(V \cdot E^2)$ whatever the capacities; Dinic [14] pushes a blocking flow through a breadth-first level graph with a current-arc pointer, in $O(V^2 \cdot E)$. Kuhn’s algorithm [15] finds maximum bipartite matchings by augmenting paths.

TextHack uses flow to analyse citations. With every citation given capacity 1, the maximum flow from one document to another equals the number of citation chains between them that share no citation, and the minimum cut is the smallest set of citations every chain depends on. Figure 4 shows the workbench’s sample: two independent chains, both depending on the two citations out of document 0.

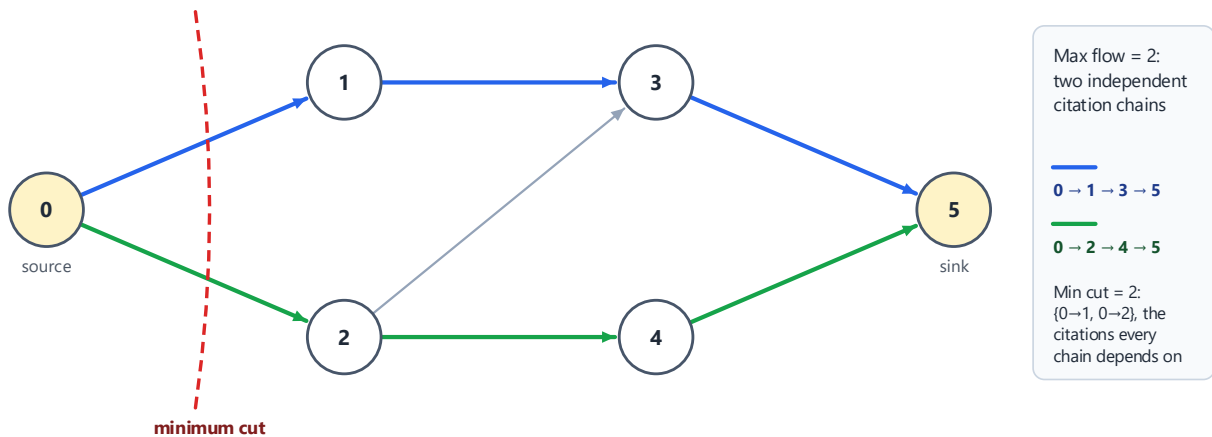


Figure 4: Citation flow: maximum flow 2 (two independent chains) equals the minimum cut $\{0 \rightarrow 1, 0 \rightarrow 2\}$

f. Approximation and Randomised Algorithms

The vertex-cover approximation takes both ends of a maximal matching, at most twice the optimum. Makespan scheduling is NP-hard: list scheduling assigns each job to the least-loaded machine and is within $2 - 1/m$ of optimal, and LPT, which first sorts jobs by length with heapsort, is within $4/3 - 1/(3m)$ [16], [17]. The syllabus listed interval scheduling here, but it is solved exactly by a greedy algorithm, so makespan scheduling replaced it as a genuine approximation problem.

Miller-Rabin [18], [19] decides primality. With a fixed set of witnesses it is exact for every 64-bit number; with k random witnesses its error is below 4^{-k} . Its modular multiplication uses the Russian-peasant method, because $(a \cdot b) \bmod m$ overflows 64 bits and a BigInteger would replace the algorithm with a library call. Universal hashing [20] bounds the collision probability by $1/m$ for integers and strings, and reservoir sampling [21] draws a uniform sample of k items in one pass over a stream of unknown length.

g. *Engine API and Query Language*

The TextHack facade gives one entry point, and a parser turns a short command language into calls on it (Table 2). The complexity registry records each algorithm's time, space and a note; it has 22 entries, because it also lists the naive reference search and both Damerau-Levenshtein variants.

Table 2: The TextHack query language

Command	Meaning	Algorithm
find "needle"	Every exact occurrence	KMP
findall "he" "she" "hers"	Every occurrence of every pattern, one pass	Aho-Corasick
fuzzy "recieve" ~2	Words within 2 edits	Damerau-Levenshtein (OSA)
similar "colour" "color"	Normalised similarity	Edit distance
prime 7919	Primality	Miller-Rabin

h. *Use in the Platform's Search*

The platform's keyword search scores each candidate document with TextHack (Figure 5, Table 3). KMP looks for the whole query as a phrase in the title and description, and Aho-Corasick checks every query term in one pass; for the document text, stored in MongoDB, the database reports the phrase and the terms it contains under the same rules. Damerau-Levenshtein (OSA) credits terms one or two edits away, the allowance growing with term length: none up to 3 letters, one edit for 4–7, two for 8 or more. A document found only by the term scan is a hit when it covers every term of a one- or two-word query, or 60% of a longer one.

Table 3: How TextHack scores a document for a keyword query

Evidence	Algorithm	Score
Whole query in the title	KMP	1.0
Whole query in the description	KMP	0.75
Whole query in the document text	Phrase match in MongoDB	0.7
Query terms, whole words	Aho-Corasick (title, description); MongoDB (text)	Coverage, up to 0.9; description and text terms count 0.6
A term one or two edits away	Damerau-Levenshtein (OSA)	Credit 0.7 or 0.5 instead of 1.0

Before searching, a QueryCorrector replaces any word that no document contains with the nearest word that one does, measured with the same OSA distance and edit allowance; ties go to the word found in more documents. A misspelled name that appears only inside a document's text, such as "Dynmo" for "Dynamo", therefore still matches, and the semantic search embeds the corrected query. Results reached this way are discounted by the typo credit and marked *Fuzzy*. The keyword score is finally fused with a semantic score from sentence embeddings, which the Database and Machine Learning courses provide.

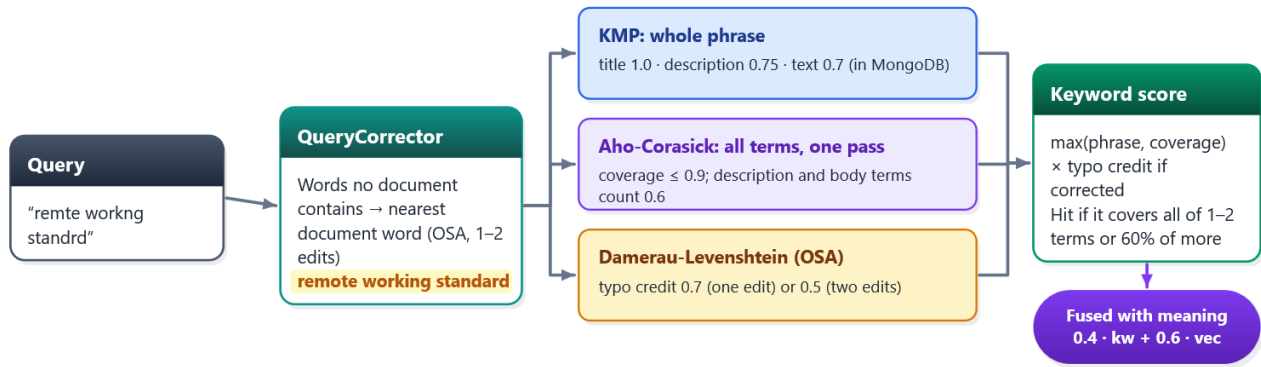


Figure 5: How a query is corrected and scored with KMP, Aho-Corasick and Damerau-Levenshtein

i. The TextHack Workbench

A workbench page lets users run the algorithms on their own input through four REST endpoints (Table 4). The controller only maps requests and responses; every result is computed by the engine. Alignment inputs are capped at 1,000 characters because alignment is $O(n \cdot m)$, the engine's rejections become 400 responses carrying its message, and every endpoint requires sign-in.

Table 4: Workbench endpoints

Endpoint	Panel	Algorithms
POST /api/texthack/pattern	Pattern search	KMP for one pattern, Aho-Corasick for several; longest repeated substring from the suffix and LCP arrays
POST /api/texthack/similarity	Similarity	Levenshtein and Damerau distances; Needleman-Wunsch and Smith-Waterman alignments
POST /api/texthack/citations	Citation flow	Dinic maximum flow; Edmonds-Karp minimum cut
GET /api/texthack/complexity	Complexity	The registry of 22 algorithms

3. EXPERIMENTS

The engine's tests need only a JDK: `run-tests.sh` compiles every source with `-Xlint:all` and runs six self-checking suites. The same script runs in the platform's GitHub Actions workflow before the backend tests, so a change that breaks an algorithm also blocks a deployment. The suites use four techniques:

- **Randomised cross-validation.** Efficient algorithms are checked against references on random inputs: 4,000 strings, 600 flow networks and 400 graphs. Strings use a 2–4 letter alphabet, so overlaps and accidental matches are common, which is where off-by-one errors surface.
- **Reproducible randomness.** A hand-written seeded generator replaces `java.util.Random`, so any failing case can be replayed from its seed.
- **Theory as a test.** Maximum flow must equal the minimum cut; approximation ratios must hold against brute-force optima; a 40,000-trial check confirms that reservoir sampling is uniform.
- **Adversarial inputs.** Carmichael numbers (561, 1105, 1729), a 2,000-character repeat that is naive search's worst case, text containing `\u0000`, CJK text, and patterns nested inside patterns.

A benchmark harness times each algorithm as the input doubles, best of five runs after three warm-up rounds, and reports the growth ratio between sizes. Exact matchers are compared on the pattern $a^{40}b$ in the text a^n , the case that separates naive search from the linear algorithms.

In the platform, the backend’s unit tests cover the scorer (18 tests), the query corrector (4) and the search service (38), and integration tests run the real search endpoint against live databases. Finally, a search inspection was run on the 315-document demo corpus on 7 October 2026: 23 queries, correctly spelled and misspelled, covering names in titles, a name and a word that appear only inside documents, full titles, single words, questions and nonsense, each in all four search modes. A case passes when the intended document is ranked first.

4. RESULTS

Correctness. All six suites pass (Table 5, Figure 6), with no compiler warnings and no `java.util` imports in algorithm code. The engine is 31 source files and 4,287 lines; the tests are 3,113 lines.

Table 5: TextHack test suites

Suite	Assertions	What it covers
StringAlgorithmTests	96	Every exact matcher against the naive reference; 4,000 random strings; overlaps, <code>\u0000</code> , CJK
SuffixAndMultiPatternTests	75	Aho-Corasick output links (“ushers”); suffix and LCP arrays against brute force
DpTests	61	Levenshtein, both Damerau variants and where they differ; both alignments
GraphTests	45	Three max-flow algorithms agree on 600 networks; max flow = min cut; matching
ApproximationAndRandomizedTests	86	Ratios against brute force; Carmichael numbers; hashing; uniform sampling
EngineTests	89	Query language, facade, citation flow, complexity registry
Total	452	0 failed, 0 warnings

```

Java 25 · TextHack (DSA-3): sh run-tests.sh: all six suites (summary lines)

$ sh run-tests.sh > run-tests.log
$ grep -E "compiled|Test Summary|All suites" run-tests.log
compiled 39 source files
Test Summary: 96 passed, 0 failed
Test Summary: 75 passed, 0 failed
Test Summary: 61 passed, 0 failed
Test Summary: 45 passed, 0 failed
Test Summary: 86 passed, 0 failed
Test Summary: 89 passed, 0 failed
All suites passed

```

Figure 6: All six suites passing (`run-tests.sh`)

Complexity. The measured growth matches each bound (Table 6, Figures 7 and 8). On the worst case, naive search grows with $n \cdot m$ while KMP and Z stay linear, 13 times faster at 160,000 characters. Aho-Corasick’s time stays nearly flat as patterns are added, while running KMP once per pattern grows with their number. Suffix-array time roughly doubles as the text doubles, as $O(n \log n)$ predicts, and Levenshtein time grows about four-fold when both strings double, as $O(n \cdot m)$ predicts. Figure 9 shows the complexity registry as users see it.

Table 6: Benchmark results (best of 5 runs after warm-up)

Experiment	Measurement
Exact search, $a^{40}b$ in a^n , $n = 160,000$	Naive 13.7 ms; KMP 1.05 ms; Z 1.03 ms; Rabin-Karp 2.8 ms
Aho-Corasick vs KMP once per pattern	5 patterns 1.3× faster; 20 patterns 4.6×; 80 patterns 15.5×
Suffix array, $n = 50,000 \rightarrow 200,000$	Growth per doubling: 1.93×, 2.27× ($O(n \log n)$)
Levenshtein, $500 \rightarrow 2,000$ characters	Growth per doubling: 5.95×, 4.57× ($O(n \cdot m)$, ideal 4×)

```

Java 25 · TextHack (DSA-3): benchmarks.Benchmark: exact and multi-pattern search

$ java -cp out benchmarks.Benchmark
=====
TextHack Benchmarks
=====
Best of 5 runs after 3 warm-up rounds.
Growth ratio compares each size against the previous one.

--- Exact single-pattern search ---
text length      naive      kmp      z      rabin-karp
20000            1644.0 us  1150.0 us  957.2 us  522.9 us
40000            3228.8 us  384.9 us  1793.5 us  868.3 us
80000            6775.1 us  525.8 us  511.9 us  1402.8 us
160000           13684.5 us 1050.9 us  1034.4 us  2844.6 us
Pattern a^40 b against a^n is the case that separates them:
naive rescans every alignment, the linear matchers do not.

--- Multi-pattern: Aho-Corasick versus repeated KMP ---
patterns      aho-corasick      kmp x k      speedup
5              5745.2 us      7499.5 us      1.3x
20             6538.3 us     30145.7 us     4.6x
80             7734.2 us     120252.9 us    15.5x
Aho-Corasick's cost is flat in the pattern count; running a
single-pattern matcher k times is not.

```

Figure 7: Benchmark: exact and multi-pattern search

```

Java 25 · TextHack (DSA-3): benchmarks.Benchmark: suffix array and Levenshtein growth

$ java -cp out benchmarks.Benchmark (continued)
--- Suffix array construction ---
text length      time      growth
25000            3371.9 us  -
50000            3461.0 us  1.03x
100000           6692.9 us  1.93x
200000           15168.8 us 2.27x
O(n log n): doubling n should roughly double the time,
plus a little for the extra doubling round.

--- Levenshtein distance ---
string length      time      growth
250                413.0 us  -
500               904.4 us  2.19x
1000              5381.5 us  5.95x
2000              24587.8 us 4.57x
O(n*m): doubling both strings quadruples the work,
so a growth near 4x is the expected shape.

```

Figure 8: Benchmark: suffix array and Levenshtein growth

Complexity				
Algorithm	Category	Time	Space	Note
Naive search	String	$O(n*m)$	$O(1)$	Reference implementation
KMP	String	$O(n+m)$	$O(m)$	No pathological input
Z-Algorithm	String	$O(n+m)$	$O(m)$	No sentinel required
Rabin-Karp	String	$O(n+m)$ expected	$O(1)$	Every hash hit verified
Aho-Corasick	String	$O(n+z)$	$O(m)$	Multi-pattern, one pass
Suffix array	String	$O(n \log n)$	$O(n)$	Prefix doubling, counting sort
LCP (Kasai)	String	$O(n)$	$O(n)$	Requires the suffix array
Levenshtein	DP	$O(n*m)$	$O(\min(n, m))$	Rolling rows
Damerau-Levenshtein	DP	$O(n*m)$	$O(n*m)$	Unrestricted variant
Damerau-Levenshtein OSA	DP	$O(n*m)$	$O(n*m)$	Restricted variant
Needleman-Wunsch	DP	$O(n*m)$	$O(n*m)$	Global alignment
Smith-Waterman	DP	$O(n*m)$	$O(n*m)$	Local alignment
Ford-Fulkerson	Graph	$O(E*maxfLow)$	$O(V)$	DFS augmenting paths
Edmonds-Karp	Graph	$O(V*E^2)$	$O(V)$	BFS augmenting paths
Dinic	Graph	$O(V^2*E)$	$O(V)$	$O(E*\sqrt{E})$ on unit capacity
Bipartite matching	Graph	$O(V*E)$	$O(V)$	Kuhn's algorithm
Vertex cover	Approximation	$O(V+E)$	$O(V)$	Ratio 2
List scheduling	Approximation	$O(n*m)$	$O(n+m)$	Ratio $2 - 1/m$
LPT scheduling	Approximation	$O(n \log n)$	$O(n+m)$	Ratio $4/3 - 1/3m$
Miller-Rabin	Randomized	$O(k*\log^3 n)$	$O(1)$	Exact for 64-bit
Universal hashing	Randomized	$O(1) / O(L)$	$O(1)$	Collision $\leq 1/m$
Reservoir sampling	Randomized	$O(n)$	$O(k)$	One pass, unknown length

Figure 9: The complexity registry in the workbench (22 algorithms)

Workbench. Figures 10 to 12 show the algorithms on users' input. Aho-Corasick finds all 11 matches of *he*, *she* and *hers* in one pass and the suffix array finds the longest repeated substring; the similarity panel shows Levenshtein charging 3 edits where Damerau charges 2, with both alignments; the citation panel reports an influence of 2 and the two bottleneck citations.

Pattern search

One pattern runs KMP. Several run Aho-Corasick, which finds every pattern in a single pass. The longest repeated substring comes from the suffix array and its LCP array. Matching is case-sensitive.

Text

she sells sea shells by the sea shore; he hears the ushers hush.

Patterns, one per line

he
she
hers

Find matches

Aho-Corasick: 11 matches

she sells sea shells by the sea shore; he hears the ushers hush.

"she" at 0-3 "he" at 1-3 "she" at 14-17 "he" at 15-17 "he" at 25-27 "he" at 39-41 "he" at 42-44 "he" at 49-51 "she" at 53-56
"he" at 54-56 "hers" at 54-58

Longest repeated substring: " sea sh"

Figure 10: Pattern search: Aho-Corasick matches highlighted, and the longest repeated substring

Similarity and alignment

Edit distance by Levenshtein and Damerau-Levenshtein, which counts a swapped pair as one edit. Needleman-Wunsch aligns the whole of both texts; Smith-Waterman finds the best matching region.

First text

recieve the report

Second text

receive the reports

Compare

Levenshtein 3 edits
Damerau 2 edits
Similarity 84%

Global

recieve the report-
receive the reports

Score 12 · identity 84% · first 0-18, second 0-19

Local

recieve the report
receive the report

Score 30 · identity 89% · first 0-18, second 0-18

Figure 11: Similarity: edit distances and global and local alignments

Citation flow

Documents are numbered from 0, and each citation points from one document to another. Dinic's maximum flow counts the separate citation chains from source to sink. Edmonds-Karp's minimum cut shows the citations every chain depends on.

Documents

Source

Sink

6

0

5

Citations, one "from to" per line

01
02
13
23
24
...

Analyse

Influence 2: 2 separate citation chains from 0 to 5.

Bottleneck citations (the minimum cut):

0 → 1 0 → 2

Source side of the cut: 0

Figure 12: Citation flow: Dinic's influence and the Edmonds-Karp minimum cut

Search inspection (Tables 7 and 8). Typo tolerance now reaches every part of a document. Before the query corrector and the coverage rule, a misspelled word that appeared only inside a document (“Dynmo”, “laptps”) was never found, semantic search found nothing for typos (“Evrest”, “budget”), and fuzzy search dropped a one-word typo matching only a description (“phising”). After them, hybrid, fuzzy and semantic search rank the intended document first in all 23 cases. Keyword mode searches exactly what was typed, so its misses are the misspellings, by design; nonsense finds nothing in every mode.

Table 7: Search inspection on the 315-document demo corpus (score of the first result; ✓ = intended document first)

Query	Hybrid	Fuzzy	Semantic	Keyword
Corvus (name in a title)	85% ✓	100% ✓	75% ✓	100% ✓
Corvos (misspelled)	73% ✓	70% ✓	75% ✓	none
Evrest (misspelled)	71% ✓	70% ✓	72% ✓	none
Dynamo (name only inside documents)	70% ✓	70% ✓	70% ✓	70% ✓
Dynmo (misspelled)	62% ✓	49% ✓	70% ✓	none
Hybird and Remte Workng Standrd	78% ✓	70% ✓	83% ✓	none
Quartely Finacial Reprot Q3	76% ✓	70% ✓	79% ✓	none
phising (word in a description)	66% ✓	52% ✓	76% ✓	none
laptps (word only inside documents)	59% ✓	49% ✓	66% ✓	none
how many days of annual leave do I get	69% ✓	47% ✓	84% ✓	47% ✓
chocolate cake recipe	none ✓	none ✓	none ✓	none ✓

Table 8: Intended document ranked first, out of 23 cases

Mode	Before	After
Hybrid (default)	21	23
Fuzzy	19	23
Semantic	18	23
Keyword (exact words only)	11	13

Figures 13 and 14 show two of these searches in the platform. The coverage rule has one visible cost: “remte workng standrd” also lists documents that contain two of the three corrected words, though they rank well below the intended standard (0.25–0.34 against 0.70).

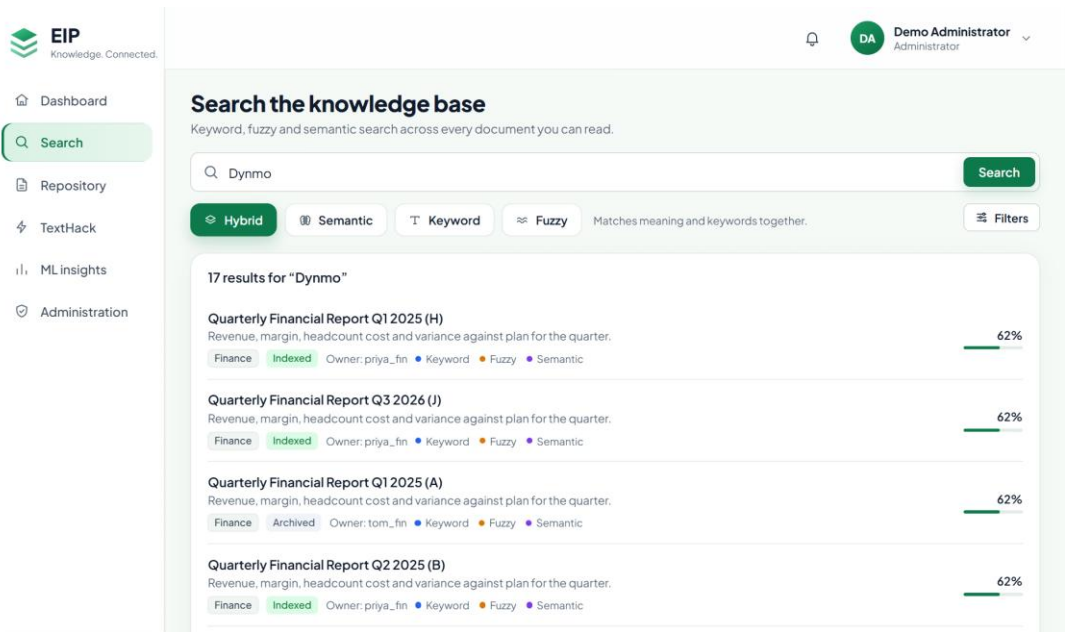


Figure 13: Hybrid search for the misspelled name “Dynmo”, found through query correction

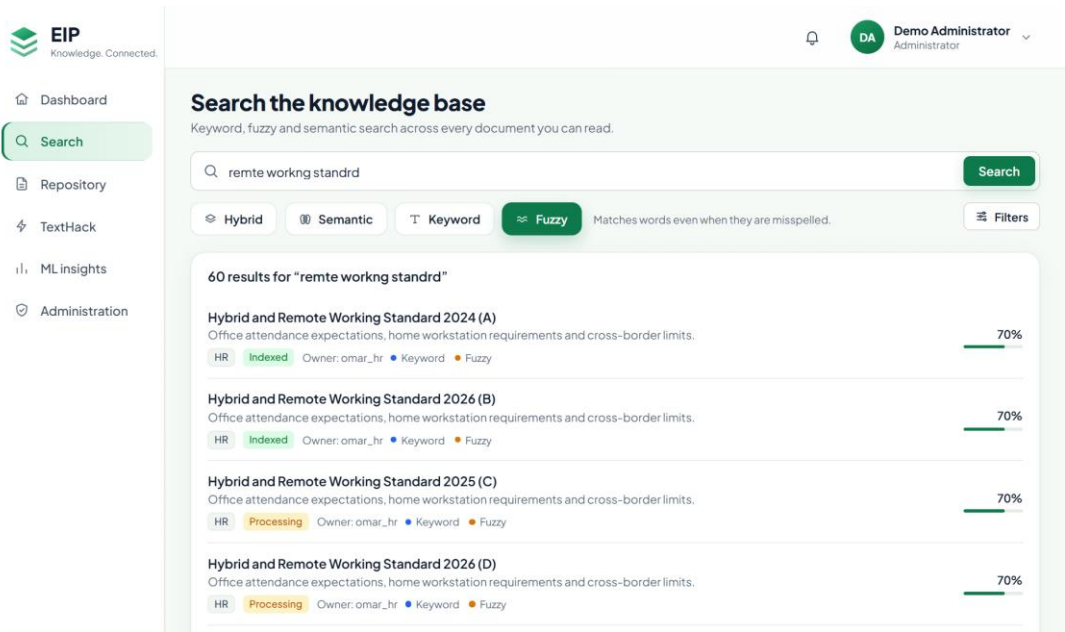


Figure 14: Fuzzy search for “remte workng standrd”: three typos, the remote-working standard first

Challenges (Table 9). Most problems came from rules that textbook code does not have to respect, and two came from measuring the search in the product rather than in isolation.

Table 9: Challenges and how they were solved

Challenge	Solution
java.util barred from algorithm code	IntList, CharMap and a seeded PRNG; counting sort for suffixes
Z-algorithm’s sentinel breaks on text containing it	Z-array of the pattern only, matched against the text: no sentinel, $O(m)$ space
Rabin-Karp hash collisions give false matches	Verify every hash hit character by character
$(a \cdot b) \bmod m$ overflows 64 bits in Miller-Rabin	Russian-peasant modular multiplication
Interval scheduling is exact, not an approximation	Makespan scheduling: list $(2 - 1/m)$ and LPT $(4/3 - 1/(3m))$
Levenshtein counts a swapped pair as 2 edits	Damerau-Levenshtein, both variants; tests pin where they differ
Aho-Corasick does not fit a one-pattern interface	Its own API, reporting which pattern matched
Fixed score threshold dropped one-word typos in descriptions	Decide hits by how many query terms match
Typos in words that appear only inside documents were never found	Correct the query against the documents’ own words with OSA

5. CONCLUSION AND FUTURE WORK

TextHack implements twenty algorithms from five families without library shortcuts, proves them with cross-validation against references, adversarial inputs and theorems written as tests, and confirms their complexity with measurements. Because the same source runs in the platform, the guarantees carry over to the product: keyword search finds every query term in one pass, tolerates typing mistakes in titles, descriptions and document text, and the workbench lets users see the algorithms at work. The 452 passing assertions, the benchmark growth rates and the 23 of 23 search inspection confirm the design.

Future work.

- An Indian-language corpus, after dump selection and a licence review.
- Candidate generation in a trigram index, so the fuzzy scan scales beyond 5,000 documents.
- BK-trees or Levenshtein automata, so query correction does not compare against every word.
- SA-IS construction for suffix arrays in $O(n)$, and push-relabel maximum flow on weighted citation graphs.
- Parallel Aho-Corasick and KMP over large texts, and a “did you mean” notice for corrected queries.

REFERENCES

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [2] D. Gusfield, *Algorithms on Strings, Trees, and Sequences*. Cambridge University Press, 1997.
- [3] D. E. Knuth, J. H. Morris, and V. R. Pratt, “Fast pattern matching in strings,” *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.
- [4] R. M. Karp and M. O. Rabin, “Efficient randomized pattern-matching algorithms,” *IBM Journal of Research and Development*, vol. 31, no. 2, pp. 249–260, 1987.
- [5] A. V. Aho and M. J. Corasick, “Efficient string matching: An aid to bibliographic search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.
- [6] U. Manber and G. Myers, “Suffix arrays: A new method for on-line string searches,” *SIAM Journal on Computing*, vol. 22, no. 5, pp. 935–948, 1993.
- [7] T. Kasai, G. Lee, H. Arimura, S. Arikawa, and K. Park, “Linear-time longest-common-prefix computation in suffix arrays and its applications,” in *Proc. Combinatorial Pattern Matching (CPM)*, 2001, pp. 181–192.
- [8] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.
- [9] F. J. Damerau, “A technique for computer detection and correction of spelling errors,” *Communications of the ACM*, vol. 7, no. 3, pp. 171–176, 1964.

- [10] S. B. Needleman and C. D. Wunsch, "A general method applicable to the search for similarities in the amino acid sequence of two proteins," *Journal of Molecular Biology*, vol. 48, no. 3, pp. 443–453, 1970.
- [11] T. F. Smith and M. S. Waterman, "Identification of common molecular subsequences," *Journal of Molecular Biology*, vol. 147, no. 1, pp. 195–197, 1981.
- [12] L. R. Ford and D. R. Fulkerson, "Maximal flow through a network," *Canadian Journal of Mathematics*, vol. 8, pp. 399–404, 1956.
- [13] J. Edmonds and R. M. Karp, "Theoretical improvements in algorithmic efficiency for network flow problems," *Journal of the ACM*, vol. 19, no. 2, pp. 248–264, 1972.
- [14] E. A. Dinic, "Algorithm for solution of a problem of maximum flow in networks with power estimation," *Soviet Mathematics Doklady*, vol. 11, pp. 1277–1280, 1970.
- [15] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, pp. 83–97, 1955.
- [16] R. L. Graham, "Bounds for certain multiprocessing anomalies," *Bell System Technical Journal*, vol. 45, no. 9, pp. 1563–1581, 1966.
- [17] R. L. Graham, "Bounds on multiprocessing timing anomalies," *SIAM Journal on Applied Mathematics*, vol. 17, no. 2, pp. 416–429, 1969.
- [18] G. L. Miller, "Riemann's hypothesis and tests for primality," *Journal of Computer and System Sciences*, vol. 13, no. 3, pp. 300–317, 1976.
- [19] M. O. Rabin, "Probabilistic algorithm for testing primality," *Journal of Number Theory*, vol. 12, no. 1, pp. 128–138, 1980.
- [20] J. L. Carter and M. N. Wegman, "Universal classes of hash functions," *Journal of Computer and System Sciences*, vol. 18, no. 2, pp. 143–154, 1979.
- [21] J. S. Vitter, "Random sampling with a reservoir," *ACM Transactions on Mathematical Software*, vol. 11, no. 1, pp. 37–57, 1985.